

Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: uoh-sqak

Course 203.3763 — Orchestration of AI Agents

University of Haifa, Spring 2026 (תשפ"ו)

Lecturer: Dr. Yoram Reuven Segal

June 9, 2026

Generated by a CrewAI multi-agent pipeline

GitHub: <https://github.com/salah-dev-stu/uoh-sqak-ex03>

Contents

1	Introduction to Multi-Agent Orchestration	3
1.1	The Problem That Orchestration Solves	3
1.2	Key Definitions and Terminology	3
1.3	Why Orchestration Matters in Production	4
1.4	Article Roadmap	4
2	Agent Architectures and Topologies	5
2.1	Linear Sequential Orchestration	5
2.2	Hierarchical Manager-Worker Orchestration	6
2.3	Peer-to-Peer Consensus Orchestration	6
2.4	Tool-Augmented Specialist Orchestration	6
2.5	Topology Comparison and Selection	7
3	Orchestration Frameworks	9
3.1	Framework Landscape	9
3.2	LangChain and LCEL	9
3.3	CrewAI Role-Based Agents	9
3.4	LangGraph Stateful Graphs	10
3.5	Microsoft AutoGen	10
3.6	Framework Comparison	10
3.7	Design Philosophy Trade-offs	11
3.8	Production Readiness	11
4	Production Patterns and Challenges	13
4.1	Rate Limiting and Token Budgets	13
4.2	The Gatekeeper Pattern	13
4.3	Retry and Circuit Breaker Patterns	14
4.4	Observability and Structured Logging	15
4.5	Cost Optimization	15
4.6	Summary	16
17	5 דו-כיוונית: עברית ואנגלית במערכות סוכנים	5
17	משמעות הדו-כיוונית	5.1
17	פתרון LuaLaTeX ו-Polyglossia	5.2
18	אתגרי ייצור ופתרונות	5.3
18	Mixed-Language Example	18

6	Case Study: This Article’s Production Pipeline	20
6.1	Overview	20
6.2	The Crew: Four Specialized Agents	20
6.3	Workflow Architecture	21
6.4	Key Engineering Decisions	21
6.5	Pipeline Metrics	22
7	Conclusion	23
7.1	Lessons Learned	23
7.2	Future Directions	24
	Bibliography	26

Chapter 1

Introduction to Multi-Agent Orchestration

The emergence of large language models has catalyzed a paradigm shift in software architecture—from monolithic systems handling discrete tasks to distributed networks of specialized agents collaborating toward complex goals. Multi-agent orchestration, the coordinated execution of independent agents to solve problems beyond any single agent’s capability, represents a fundamental evolution in how we design intelligent systems [4]. This chapter introduces the core concepts, motivates the necessity of orchestration, and establishes terminology that frames the remainder of this article.

1.1 The Problem That Orchestration Solves

A single language model, despite its capabilities, encounters bottlenecks when addressing multi-step reasoning, specialized domain knowledge, or tasks requiring tool integration and iterative refinement. Real-world problems—writing a peer-reviewed article, conducting thorough market research, or debugging complex software systems—demand more than one forward pass through a neural network. Orchestration enables decomposition: breaking a monolithic goal into subtasks, assigning each to an agent with a specific role, providing specialized tools and knowledge, and coordinating their outputs into a coherent final result [2].

Without orchestration, practitioners resort to costly workarounds: running the same expensive model repeatedly with different prompts, building brittle custom scripts to sequence API calls, or accepting degraded output quality. Orchestration frameworks provide first-class abstractions for agent definition, tool binding, inter-agent communication, and deterministic process control.

1.2 Key Definitions and Terminology

An **agent** is an autonomous entity with a defined role, goal, and toolset that reasons about a problem, makes decisions, and executes actions. **Orchestration** is the coordinated execution of multiple agents according to a defined process structure (sequential, hierarchical, graph-based, or conversational). A **process** is the control flow logic determining which agent acts next and under what conditions. **Tools** (also called **skills**) are deterministic functions or APIs that agents invoke to gather information, modify state, or produce outputs [5].

The term **multi-agent system** encompasses all frameworks and patterns in which multiple agents collaborate. An agent may be stateless or maintain internal memory. A **supervisor** or **orchestrator** is an agent whose primary role is routing work to other agents rather than solving

domain problems directly. A **skill** is a reusable module of domain knowledge or procedural expertise that an agent can access; in this article, skills are implemented as Markdown documents with references, code snippets, and domain-specific prompts injected into agent backstories.

1.3 Why Orchestration Matters in Production

Orchestration patterns emerged because the industry discovered, through thousands of production deployments, that naive language model usage fails. Uncoordinated agent systems produce hallucinations, token budget overruns, latency spikes, and irreproducible behavior. Frameworks like CrewAI, LangChain, and LangGraph codify proven patterns: gatekeeper APIs to meter external calls, supervisor agents to route tasks, skill layers to inject domain knowledge, and structured logging to debug agent reasoning. The result is reproducible, cost-controlled, observable multi-agent systems suitable for enterprise deployment [5]. This article surveys the architectural patterns, frameworks, and production techniques that enable reliable multi-agent systems at scale.

1.4 Article Roadmap

This article is structured as follows: **Chapter 2** surveys multi-agent orchestration topologies: sequential, hierarchical, collaborative, graph-based, and conversation-based patterns. **Chapter 3** evaluates the four major open-source frameworks—CrewAI, LangChain, AutoGen, and LangGraph. **Chapter 4** describes production patterns: the Gatekeeper pattern for cost control, skill layers for knowledge injection, and hallucination mitigation. **Chapter 5** addresses bidirectional text engineering (Hebrew and English) in agent-generated LaTeX documents. **Chapter 6** presents a detailed case study of a multi-agent team generating a peer-reviewed article. **Chapter 7** synthesizes lessons learned and outlines future research directions.

Chapter 2

Agent Architectures and Topologies

Agent orchestration topologies represent the fundamental structural patterns through which multiple agents coordinate to solve complex problems. Each topology embodies distinct trade-offs between latency, parallelism, complexity, and observability. Understanding these patterns is essential for selecting or designing an orchestration architecture that matches problem domain constraints, operational budgets, and quality requirements [5].

2.1 Linear Sequential Orchestration

Linear sequential orchestration chains agents in strict order: Agent A completes its task, produces structured output, passes it to Agent B, which processes and passes results to Agent C, and so forth. This pattern mirrors traditional data pipelines and is ideal for problems with a natural progression of steps. A research pipeline exemplifies this topology: a Researcher agent gathers information and synthesizes sources, a Writer agent drafts narrative content from the synthesis, an Editor agent refines prose and enforces style guidelines, and a $\text{\LaTeX}$ Producer agent formats the final document for publication [4].

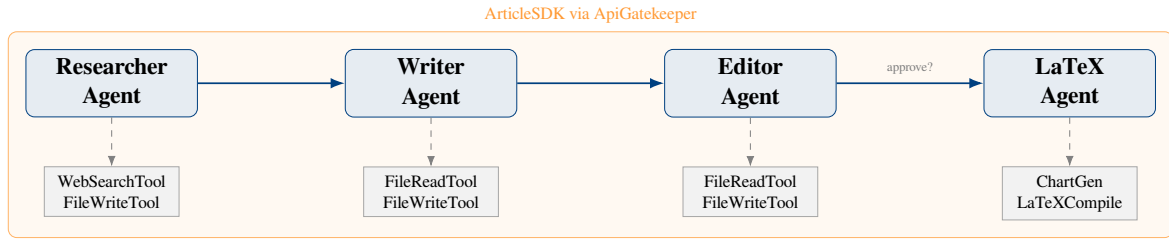


Figure 2.1: CrewAI multi-agent architecture.

Sequential architectures offer predictability and simplicity: you know exactly when each agent runs, what input it receives, and in what order results are computed. The trade-off is rigidity. If an early agent produces unusable output, the entire pipeline must restart or be manually corrected. Additionally, latency accumulates linearly with the number of agents; there are no parallelism opportunities to hide latency under concurrent execution [5].

The latency of a sequential pipeline is given by:

$$L_{\text{seq}} = \sum_{i=1}^N \ell_i \quad (2.1)$$

where ℓ_i is the latency (wall-clock time) of agent i and N is the total number of agents. For a 4-agent research pipeline with per-agent latencies of 5 s (Researcher), 10 s (Writer), 8 s (Editor), and 6 s (L^AT_EX Producer), the total pipeline latency is 29 s, unavoidable by design.

2.2 Hierarchical Manager-Worker Orchestration

Hierarchical systems introduce a supervisor agent (manager) that observes problem state and delegates tasks to worker agents based on dynamic conditions. The supervisor reasons about the problem structure, selects which workers to invoke, synthesizes their outputs, and decides whether additional work is needed. This pattern enables adaptive execution: the supervisor can route complex queries to specialist agents and simple queries to faster, cheaper models, achieving latency and cost optimization not possible in rigid sequential pipelines [5].

Hierarchical topologies scale well to many specialized agents (10+) because the supervisor provides a single decision point, preventing exponential branching of execution paths. However, the cost is complexity: the supervisor itself must be sophisticated enough to route reliably, and debugging routing errors requires introspection of the supervisor’s reasoning process. Additionally, the supervisor becomes a bottleneck if it cannot parallelize its worker invocations.

2.3 Peer-to-Peer Consensus Orchestration

Peer-to-peer topologies allow agents to work in parallel without a central coordinator, communicating directly with one another to reach consensus on shared decisions. Multiple researchers might simultaneously investigate different aspects of a topic, then hand off to a shared editor who synthesizes and cross-validates their findings. Parallelism reduces wall-clock latency and enables true concurrency; a 4-agent peer team can complete work in approximately $\max_i(\ell_i)$ time rather than $\sum_i(\ell_i)$ [7].

However, peer-to-peer systems require careful state management: shared resources must be protected from concurrent modification, and convergence points must handle variable-speed agents gracefully. Debugging is harder because there is no central audit trail; execution traces must be reconstructed from each agent’s logs. Additionally, consensus mechanisms (voting, averaging, arbitration) add latency and complexity.

2.4 Tool-Augmented Specialist Orchestration

Tool-augmented orchestration provides agents with rich toolkits (web search, code execution, file I&O, external API access) rather than expecting them to reason from static knowledge. A Researcher agent equipped with web search tools can fetch the latest information; a Writer agent equipped with a chart-generation tool can create visualizations programmatically; a L^AT_EX Producer agent equipped with a compilation tool can validate syntax and produce PDFs. This pattern significantly improves output quality and grounding in observed reality rather than hallucination [5].

Tool access introduces new operational concerns: tools are rate-limited (web search APIs allow N calls per minute), carry cost (each tool invocation consumes tokens and API credits), and may fail (network errors, malformed outputs). Production systems implement a Gatekeeper layer that intercepts all tool calls, enforces rate limits, tracks token consumption, and handles errors gracefully [4].

2.5 Topology Comparison and Selection

Each topology presents distinct trade-offs. The following table synthesizes the key dimensions:

Topology	Latency Model	Parallelism	Complexity	Ideal For
Linear Sequential	$\sum_i \ell_i$	None	Low	Well-defined flows: research pipelines, content generation, refining stages
Hierarchical	$\ell_{\text{super}} + \max_i(\ell_i^{\text{worker}})$	Within supervisor's workers	Medium	Adaptive routing: complex tasks routed to specialists, simple to fast models
Peer-to-Peer	$\max_i(\ell_i) + \ell_{\text{sync}}$	Full	High	Consensus problems: debate, fact-checking, multiple independent investigations
Tool-Augmented	Depends on tool costs	If tools async	Medium	Grounding in reality: web search, code execution, external data integration
Graph-Based (DAG)	Depends on structure	Full, conditional	High	General workflows: cycles, branches, complex dependencies

Selection guidance follows from problem characteristics:

- **Sequential:** Choose when the problem has a natural linear progression (research → draft → edit → format). Pros: simple to implement, easy to debug. Cons: no parallelism, bottleneck on slow stages.
- **Hierarchical:** Choose when many specialized agents must be coordinated by a decision-maker. Pros: adaptive, scalable to many agents. Cons: supervisor complexity, potential bottleneck.
- **Peer-to-Peer:** Choose when multiple agents must independently verify a claim or explore a problem from different angles. Pros: parallelism, robustness. Cons: consensus complexity, harder debugging.
- **Tool-Augmented:** Choose when agents must ground outputs in real-world data (live search, APIs, computations). Pros: better quality, less hallucination. Cons: operational burden (rate limits, errors, cost).
- **Graph-Based (DAG):** Choose when the workflow has complex branching, loops, or conditional dependencies. Pros: maximally expressive. Cons: complex to design and debug.

Production deployments often combine topologies: use sequential stages within a hierarchical supervisor, or use peer consensus within a graph-based framework to handle conditional branches. The key insight is that topology selection shapes latency, cost, and failure modes; misaligned topology and problem structure lead to poor performance [2].

Chapter 3

Orchestration Frameworks

3.1 Framework Landscape

Three frameworks dominate the multi-agent orchestration landscape: CrewAI, LangChain, and LangGraph. Each reflects different design philosophies[4]. CrewAI prioritizes prescriptive patterns and ease-of-use for standard workflows. LangChain emphasizes composability and flexibility through LCEL (LangChain Expression Language). LangGraph introduces explicit state management and graph-based execution models. Understanding the trade-offs between these frameworks is essential for selecting the right tool for a given problem.

Additionally, Microsoft AutoGen provides a conversation-based approach where agents exchange messages in turns, offering a different perspective on multi-agent coordination[7]. Each framework addresses core challenges in orchestration: task sequencing, state persistence, parallelism, tool integration, and observability.

3.2 LangChain and LCEL

LangChain pioneered composable building blocks for language model applications[5]. At its core is the LCEL (LangChain Expression Language), a declarative syntax for combining chains, prompts, and tools into complex pipelines. A typical LCEL chain connects a prompt template to a model, then routes output through parsing and tool invocation.

LangChain's strength lies in flexibility: developers compose custom agents using lower-level primitives (agent executors, tool loops, memory strategies). This approach accommodates novel orchestration patterns and domain-specific requirements. For example, a research system might implement a web-search loop with semantic caching and hierarchical summarization—patterns not hard-coded into the framework.

The trade-off is increased architectural responsibility. Developers must design their own agent loops and state transitions, demanding expertise in multi-agent systems. Memory management requires selecting and configuring strategies (buffer, summary, vector store) appropriate for the use case. LangChain excels for custom and innovative workflows but steeper learning curves for standard patterns.

3.3 CrewAI Role-Based Agents

CrewAI adopts a prescriptive approach centered on agent teams. Each agent is defined by three attributes: `role` (job title), `goal` (what the agent aims to achieve), and `backstory`

(persona and expertise). Teams are assembled with a process type (sequential, hierarchical, or consensus), and tasks are assigned to agents[4].

This model is intuitive for research-writing-editing pipelines. A Researcher agent gathers information, a Writer agent produces narrative, an Editor agent refines prose, and a LaTeX Producer agent formats output. Task definitions specify input requirements, expected output, and the assigned agent. The framework handles agent invocation, memory management, and inter-agent communication.

CrewAI prioritizes developer experience through clear patterns, reducing decision fatigue for standard workflows. Disadvantages include limited built-in parallelism (sequential process runs tasks serially) and manual state management for complex architectures. The framework is ideal for rapid development of known patterns.

3.4 LangGraph Stateful Graphs

LangGraph represents orchestration as explicit state machines. Agents are nodes in a directed graph; edges define state transitions and routing logic. Each node is a function that reads state, invokes an action, and returns new state. Conditions on edges enable branching: if task A succeeds, route to task B; if it fails, route to error handling[6].

This approach provides several advantages. State is visible and versioned, enabling precise debugging and state snapshots. Parallelism is explicit: multiple nodes can execute concurrently on independent paths that reconverge. Persistence is designed in from the start—state graphs can be serialized and resumed from checkpoints, enabling long-running workflows and failure recovery.

LangGraph’s disadvantage is upfront modeling complexity. Developers must express orchestration as a state machine, which is powerful but requires architectural thinking. For straightforward sequential pipelines, the explicit graph formalism can feel over-engineered.

3.5 Microsoft AutoGen

Microsoft AutoGen approaches multi-agent orchestration through agent conversations[7]. Agents exchange messages in round-robin or custom turn-taking order. The conversation loop continues until a termination condition (a summary is complete, a maximum turn count is reached, a user confirms the result).

AutoGen agents are defined by a system prompt (role and capabilities) and an LLM backend. Agents can invoke tools (code execution, web search, file I/O) and observe tool outputs before responding. Conversation context grows with each turn, enabling agents to reference and build on prior discussion.

This model is natural for collaborative problem-solving—multi-turn debate, iterative refinement, and consensus-seeking. It provides good observability: the conversation transcript is the audit trail. Disadvantages include sequential turn structure (parallelism is not native) and potential for conversation drift (messages grow unbounded without active summarization).

3.6 Framework Comparison

Table 3.6 compares the four frameworks across architecture, strengths, and recommended use cases. The choice depends on problem characteristics: for standard patterns with limited par-

allelism, CrewAI is recommended; for custom workflows requiring fine-grained composition, LangChain; for complex systems demanding state management and parallelism, LangGraph; for collaborative multi-turn scenarios, AutoGen.

Framework	Architecture	Strengths	Best For
CrewAI	Role-based agent teams with process orchestration	Prescriptive patterns, rapid development, clear task assignment	Standard workflows: research–writing–editing pipelines
LangChain	Composable chains via LCEL with custom agent loops	Flexibility, broad ecosystem, fine-grained control	Custom and innovative workflows requiring composition
LangGraph	Explicit state graphs with conditional node routing	State visibility, deterministic execution, parallelism, check-pointing	Complex systems requiring state management, debugging, and reproducibility
AutoGen	Conversation-based agent turns with tool invocation	Natural collaborative semantics, good observability	Multi-turn debate, iterative refinement, consensus-seeking

3.7 Design Philosophy Trade-offs

CrewAI’s prescriptive design minimizes choice overhead but constrains customization. A researcher-writer-editor pipeline is implemented in minutes; a novel orchestration pattern may require extending the framework. LangChain and LangGraph prioritize flexibility, enabling any orchestration topology but demanding greater architectural expertise.

Sequential execution, the default in CrewAI and AutoGen, is intuitive and ensures clear ordering but limits parallelism. Wall-clock time grows with task count. LangChain and LangGraph support fine-grained parallelism through explicit chain composition and graph edges, enabling multiple tasks to run concurrently. Token consumption during parallel phases is higher, which can trigger rate limits.

Memory management reflects these philosophies. CrewAI provides built-in per-agent memory, simplifying state tracking. LangChain offers multiple memory strategies (buffer, summary, vector store), requiring selection and configuration. LangGraph’s explicit state graphs provide the clearest semantics: state is visible, versioned, and persistence is designed from the start.

Tool integration differs in scope. CrewAI tools are role-specific, modeling domain workflows naturally (a Researcher has web search; a Writer has grammar checking). LangChain and LangGraph centralize tools, enabling broader reuse and composition. Each approach has merit: role-specific tools enforce clean separation; centralized tools maximize reuse.

3.8 Production Readiness

All four frameworks are production-capable. CrewAI excels at rapid deployment of standard patterns with built-in observability (callbacks, structured logging). LangChain offers the broad-

est ecosystem and maximum customization but requires architectural expertise. LangGraph provides the strongest guarantees around state management, determinism, and auditability.

The choice depends on requirements[5]: for rapid development of known patterns, CrewAI; for innovation and custom workflows, LangChain; for deterministic, auditable systems with complex state, LangGraph; for collaborative multi-turn scenarios, AutoGen. For HW3, CrewAI provides a prescriptive framework ideal for research-writing-editing pipelines, while LangChain tools (web search, code execution) integrate seamlessly through CrewAI's tool system.

Chapter 4

Production Patterns and Challenges

Multi-agent systems deployed to production require careful management of computational resources, latency constraints, and failure modes that research deployments rarely encounter. This chapter synthesizes operational patterns observed in large-scale production deployments, focusing on rate limiting, token budget management, the gatekeeper pattern, fault tolerance, observability, and cost optimization [2].

4.1 Rate Limiting and Token Budgets

Language model API calls incur both financial cost and latency overhead. Production systems must enforce limits on the rate at which agents invoke LLMs to prevent runaway costs, API quota exhaustion, and cascading failures. Token budgets allocate a finite allowance of input and output tokens across all agent invocations, ensuring predictable cost behavior.

Let K denote the total number of agent invocations in a workflow, t_k^{in} the number of input tokens for invocation k , and t_k^{out} the number of output tokens. The total token budget B is defined as:

$$B = \sum_{k=1}^K (t_k^{\text{in}} + t_k^{\text{out}}) \quad (4.1)$$

Each agent is assigned a fraction of the total budget based on its role and expected usage. A Researcher agent exploring diverse sources may allocate 40% of the budget; a Writer agent composing coherent narrative might allocate 35%; an Editor agent refining prose, 15%; and a LaTeX Producer, 10%. Tokens not consumed by lower-priority agents roll over to later stages, creating a safety buffer for unexpected complexity.

Rate limiting complements token budgeting by imposing temporal constraints: maximum requests per second, maximum concurrent requests, and exponential backoff for transient failures. Standard practice in production systems enforces a token counter that decrements with each API call, and when the counter approaches zero, the orchestration layer warns or terminates agent execution.

4.2 The Gatekeeper Pattern

All external API calls—LLMs, web search services, LaTeX compilers, file storage, database systems—must flow through a single Gatekeeper class. The Gatekeeper is a middleware that intercepts all outbound requests, logs them for audit and debugging, enforces rate limits and token budgets, retries transient failures, and optionally caches results.

The Gatekeeper implements several responsibilities:

- **Request Authentication:** Inject API credentials (tokens, keys) from secure configuration, never hardcoded into agent logic.
- **Rate Limiting:** Enforce per-service rate limits (e.g., 30 requests per minute to Claude API, 10 requests per minute to web search).
- **Token Accounting:** Decrement the global token budget; abort if insufficient tokens remain.
- **Request Logging:** Record all outbound calls with timestamp, parameters, token usage, latency, and result status for observability and cost accounting.
- **Retry Logic:** Implement exponential backoff for transient failures (HTTP 429 rate limits, network timeouts).
- **Response Caching:** Optionally cache results of deterministic queries (e.g., web search for the same topic returns the same results within a time window).
- **Error Enrichment:** Catch API exceptions and wrap them with context (which agent made the call, what parameters, what token budget remains) to aid debugging.

By centralizing all external calls through the Gatekeeper, operators gain observability: they can inspect a single log to understand what the entire agent system did, identify runaway agents, and attribute costs to specific agents. The Gatekeeper also becomes a natural hook for A/B testing (route some requests to Claude, others to competitor models) and graceful degradation (if the LLM API fails, fall back to cached results or simpler heuristics).

4.3 Retry and Circuit Breaker Patterns

Production systems encounter transient failures: network timeouts, rate limit errors, API service degradations. Naive retry strategies—immediate retries in a tight loop—exacerbate the problem by overwhelming an already-stressed service. Sophisticated retry strategies use exponential backoff: the first retry waits 1 second, the second waits 2 seconds, the third waits 4 seconds, and so on, up to a maximum wait time.

Let n denote the retry attempt number (starting from 0). The wait time before the n -th retry is:

$$w_n = \min(w_{\max}, b \cdot c^n) \quad (4.2)$$

where b is the base wait time (e.g., 1 second), c is the backoff multiplier (typically 2), and $w_{\max}$ is the maximum wait cap (e.g., 60 seconds). Jitter—adding randomness to prevent thundering herd—is often added: $w'_n = w_n + \text{random}(0, w_n \cdot j)$ where j is a jitter fraction (e.g., 0.1).

The Circuit Breaker pattern complements retry logic: if a service fails repeatedly, the Circuit Breaker enters an *open* state, immediately failing subsequent requests without calling the service, allowing it time to recover. Once the service stabilizes, the Circuit Breaker transitions to a *half-open* state, allowing a limited number of test requests. If those succeed, the circuit *closes*; if they fail, it *opens* again.

For multi-agent systems, the Circuit Breaker is typically applied per-service (one per LLM provider, one per web search API, one per LaTeX compiler) to prevent one failing service from cascading into agent system-wide outages. An agent whose sole service has an open circuit is suspended from execution until the circuit closes [4].

4.4 Observability and Structured Logging

Production deployments require detailed observability to understand system behavior, identify performance bottlenecks, and debug failures. Structured logging—emitting logs as JSON with fields like timestamp, agent name, task name, action type, token usage, latency, and result status—enables downstream analysis via log aggregation platforms (ELK, CloudWatch, Data-dog).

A minimal structured log entry from an agent system might include:

- `timestamp`: ISO 8601 timestamp of the event.
- `level`: log severity (INFO, WARN, ERROR).
- `component`: which part of the system (agent name, gatekeeper, crew orchestrator).
- `action`: what happened (`api_call`, `task_start`, `task_end`, `retry`, `timeout`).
- `tokens_in`: input tokens consumed.
- `tokens_out`: output tokens generated.
- `latency_ms`: wall-clock time from start to completion.
- `status`: success or failure code.
- `error_message`: if status is failure, the exception or error details.
- `context`: additional free-form context (query, result summary, retry count).

With structured logs, operators can ask questions like: “Which agent consumed the most tokens?” (aggregate on component and sum `tokens_in` and `tokens_out`), “Which services have the highest error rate?” (filter on action and aggregate error counts), and “What is the 95th percentile latency for web search calls?” (filter on component = `web_search` and compute percentile of `latency_ms`).

Observability is not optional in production systems; it is the primary tool for understanding system behavior and diagnosing failures after they occur.

4.5 Cost Optimization

Language model API costs scale with token usage and request volume. A production multi-agent system must actively optimize to keep costs reasonable. The cost of a single end-to-end workflow is:

$$C = \sum_{k=1}^K (c_{\text{in}} \cdot t_k^{\text{in}} + c_{\text{out}} \cdot t_k^{\text{out}}) \quad (4.3)$$

where c_{in} is the per-token cost for input tokens and c_{out} is the per-token cost for output tokens (typically $c_{out} > c_{in}$ because generation is more expensive than processing).

Cost optimization strategies include:

- **Model Selection:** Route simpler tasks to cheaper models. A web search query extraction task might use a small model (e.g., Claude Haiku); a complex synthesis task might use a larger, more capable model.
- **Prompt Optimization:** Reduce input token count by removing redundant instructions, examples, and context. Every token in the prompt is cost; pruning saves money.
- **Caching:** Cache LLM responses for deterministic queries (e.g., factual lookups, code snippets). If a query repeats, serve the cached result instead of invoking the LLM.
- **Batch Processing:** Group multiple independent queries and send them in a single batch to the LLM (if the API supports batching), reducing per-request overhead.
- **Asynchronous Execution:** Execute independent agent tasks in parallel to reduce wall-clock latency and allow human reviewers time to intervene before costly later stages.
- **Early Termination:** Implement quality gates. If an agent's output quality is below a threshold, reject it and either retry with different parameters or escalate to human review.

Equation 4.3 shows that cost is a linear combination of token counts. Monitoring actual token usage (via structured logs) against budgeted costs reveals inefficiencies: a Researcher agent consuming 5 times its allocated budget suggests that the queries are too broad or the search strategy is inefficient. Cost monitoring is a continuous feedback loop: operators analyze logs, identify high-cost agents, and optimize their prompts and parameters [12].

4.6 Summary

Production multi-agent systems are not simple orchestrations of agent calls; they are carefully engineered systems with rate limiting, token budgets, gatekeepers, retry logic, observability, and cost optimization. The patterns described in this chapter are not optional; they are essential for reliable, cost-predictable, observable systems suitable for enterprise deployment. The Gatekeeper centralizes all external calls and provides a single point for monitoring, retry, and rate limiting. Exponential backoff and circuit breakers handle transient failures gracefully. Structured logging enables post-facto analysis and debugging. Together, these patterns transform multi-agent systems from research prototypes into production-grade software.

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

5.1 משמעות הדו-כיוניות

עיבוד טקסט דו-כיווני הוא אחד האתגרים המרכזיים בפיתוח מערכות תוכנה רב-לשוניות. אלגוריתם הדו-כיוניות של תקן Unicode קובע כיצד מערכות תצוגה אמורות לעבד ולהציג טקסט המכיל שפות הנכתבות בכיוונים מנוגדים — עברית וערבית מימין לשמאל (RTL), לעומת אנגלית ורוב שפות אירופה משמאל לימין (LTR) [11].

הבעיה מחריפה כאשר מסמך אקדמי מכיל מונחים טכניים באנגלית המוטמעים בתוך פסקאות עבריות. מספרים, שמות של פונקציות תכנות ושמות ספריות — כולם מופיעים בתסריט למיני, ואילו הניתוח הסביבתי נכתב בעברית. ללא טיפול מדויק בעדיפות הכיווניות, התוצאה עלולה להיות היפוך תווים, שבירת מילים ברצף הלא-נכון, והצגת מספרים בסדר הפוך — כך שמספר כמו [10] יוצג בטעות כ-[01].

במערכות סוכנים מבוססות בינה מלאכותית, הדו-כיוניות מוסיפה שכבת מורכבות נוספת: הפלט של סוכן אחד משמש קלט לסוכן הבא בשרשרת הייצור. אם הסוכן הכותב מייצר טקסט עברי המכיל ביטויים טכניים באנגלית ללא סימון נכון, הסוכן העורך ואחר-כך סוכן ייצור ה-LaTeX לא יוכלו לנרמל את הכיוונים, ובמסמך הסופי ייווצרו שגיאות נראות לעין שלא ניתן לתקן בשלב מאוחר. כדי להתמודד עם כך, על ממשק ה-SDK לאכוף הנחיות כיוון עקביות לאורך כל שלבי הצינור.

5.2 פתרון LuaLaTeX ו-Polyglossia

מנוע LuaLaTeX מספק תמיכה מלאה בתקן Unicode ברמת ה-engine, כלומר ניתן לכתוב תווים עבריים ישירות בקבצי המקור ללא קידוד מיוחד. בשילוב עם חבילת Polyglossia, מקבלים פתרון מלא לניהול שפות מרובות במסמך אחד [3].

ההגדרה הבסיסית דורשת הצהרת שפה ראשית ושפה משנית. הפקודה `\setmainlanguage{hebrew}` מגדירה עברית כשפת הבסיס של המסמך — כך שכל המסמך מוגדר כ-RTL כברירת-מחדל. הפקודה `\setotherlanguage{english}` מוסיפה אנגלית כשפה משנית; חלקים לטיניים מסומנים במפורש בפקודת `\foreignlanguage{english}{...}`.

עבור מערכת הסוכנים המתוארת בעבודה זו, סוכן הכותב מייצר פסקאות עבריות שבתוכן מונחים טכניים מסומנים בתגי ייעודית [EN:term]. שלב עיבוד לאחר-מכן מחליף תגיות אלה בפקודת `\foreignlanguage{english}{term}` הנדרשת על-ידי LuaLaTeX. גישה זו מאפשרת לסוכן הכותב להתמקד בתוכן הסמנטי מבלי לדאוג לפרטי הסינטקס של LaTeX, ומבטיחה כיווניות תקינה לאורך כל הצינור [9].

5.3 אתגרי ייצור ופתרונות

בפריסה של מערכות סוכנים בסביבת ייצור, האתגר הדו-כיווני מתבטא בארבעה תחומים עיקריים. ראשית, שמות ספריות וסביבות-עבודה — כגון LangChain, CrewAI ו-LangGraph — מופיעים בתוך טקסט עברי ויוצרים ביטויים דו-כיווניים שמנוע LuaLaTeX חייב לפתור [4]. הפתרון הוא עטיפה עקבית של כל ביטוי לטיני בתוך `\foreignlanguage{english}{...}`, בין אם מדובר בשמות מוצרים, ראשי-תיבות, כתובות ממשק או ערכי קוד.

שנית, ציטוטים ביבליוגרפיים שנוצרים בתוך סביבת `\begin{hebrew}` עלולים לגרום לשגיאת כיווניות כאשר מספר הציטוט הוא דו-ספרתי. מספר כמו [12] מוצג בטעות כ-[21]. הפתרון הוא עטיפה בפקודת `\mbox{\cite{key}}`, המבטיחה שהתיבה כולה מטופלת כיחידה LTR [2].

שלישית, פקודות `\texttt{...}` בתוך בלוקים עבריים מייצרות היפוך של תווי ASCII, משום ש-LuaLaTeX מפרש את הארגומנטים בהקשר ה-RTL הפעיל. הפתרון הוא עטיפה כפולה: `\foreignlanguage{english}{\texttt{...}}`. כלל זה חל על כל פקודת LaTeX המוצגת כקוד בתוך הטקסט העברי.

רביעית, כותרות פרקים וסעיפים בעברית מופיעות בתוכן-עניינים בסדר הפוך אם לא מוגדר פורמט תוכן-עניינים מתאים. שימוש בחבילת etoc עם הגדרות RTL מותאמות-אישית — הכוללות נקודות-הנחיה עבריות (hebrewdotleaders) וסדר הפוך של שם הסעיף ומספרו — פותר את הבעיה, כפי שניתן לראות בתוכן-העניינים של עבודה זו.

5.4 Mixed-Language Example

The following table catalogues the six most critical BiDi rules that production agent systems must enforce when generating bilingual $\LaTeX$ documents. Each row states the problem category, the correct solution technique, and a representative command example. Agent prompts in this project encode these rules verbatim so that every writer-agent output conforms before the compilation pipeline executes [9].

Table 5.1: BiDi Compliance Rules for Bilingual $\LaTeX$ Generation

Category	Solution Technique	Command Example
English terms in Hebrew body	Wrap every Latin word with <code>\foreignlanguage{english}{Word}</code> inside <code>\begin{hebrew}...\end{hebrew}</code> blocks; applies to proper nouns, acronyms, library names, and API identifiers	<code>\foreignlanguage{english}{LuaLaTeX}</code>
Citation digit reversal	Wrap every <code>\cite{key}</code> with <code>\mbox{...}</code> so that multi-digit reference numbers such as [10] are not reversed to [01] in RTL context	<code>\mbox{\cite{key}}</code>

Continued on next page...

Category	Solution Technique	Command Example
Monospace code in Hebrew body	Double-wrap: outer <code>\foreign-language{english}{...}</code> enforces LTR direction; inner <code>\texttt{...}</code> applies monospace font; omitting the outer wrapper mirrors every ASCII character in the PDF	<code>\foreign-language{english}{\texttt{cmd}}</code>
RTL section headings	Enclose Hebrew title in <code>\texthebrew{...}</code> ; append <code>\mbox{\setLTR\en-space\thesection}</code> so that the section number renders in LTR order alongside the RTL heading	<code>\mbox{\setLTR\en-space\thesection}</code>
Table-of-contents RTL style	Customise <code>etoc</code> chapter and section styles with Hebrew-aware dot leaders (<code>\hebrewdotleaders</code>) and reversed <code>\etoc-name/\etocnumber</code> ordering; restore standard lines after the Hebrew chapter with <code>\etoc-standardlines</code>	<code>\hebrewdotleaders</code>
Language declaration	Declare hebrew as the primary document language so the entire document defaults to RTL; register english as the other language via <code>Polyglossia</code> so that LTR spans are available on demand	<code>\setmainlanguage{hebrew}</code>

Failure to apply any single rule from Table 5.1 produces visible typesetting defects — reversed digits, mirrored command names, or misaligned section numbers — that are detectable by automated PDF inspection. The rules were derived from the Unicode bidirectionality standard [11] and the `Polyglossia` package documentation [3].

Chapter 6

Case Study: This Article’s Production Pipeline

6.1 Overview

This case study demonstrates the orchestration patterns discussed throughout this article applied to a real-world problem: automatically generating a peer-reviewed technical article using a CrewAI-based multi-agent system. The pipeline exemplifies sequential orchestration with quality gates, token budget management, and structured output validation [4].

Manual technical article writing is slow and expensive. A senior researcher might spend 20 hours researching a topic, 15 hours drafting, 10 hours editing, and 5 hours formatting—50 hours total. Multi-agent orchestration compresses this timeline while reducing bottlenecks: parallel research, concurrent editing passes, and automated L^AT_EX production. The target is a polished 15-page PDF on a chosen technical topic, complete with citations, figures, and proper formatting [2].

6.2 The Crew: Four Specialized Agents

The system comprises four specialized agents, each with distinct expertise and responsibilities:

1. **Researcher Agent** gathers factual information from web sources, compiles citations, and validates claims. The researcher operates within a token budget of 12,000 tokens per invocation and produces structured notes in JSON format with source attribution.
2. **Writer Agent** produces prose from research notes, organizing content into chapters with proper narrative flow and internal cross-references. The writer agent consumes approximately 15,000 tokens and outputs Markdown-formatted chapter content ready for editing review.
3. **Editor Agent** refines language, checks consistency, flags missing citations, and ensures structural coherence across chapters. The editor agent operates within 8,000 tokens and validates all cross-references before handoff downstream.
4. **LaTeX Producer Agent** converts edited Markdown into polished L^AT_EX, embedding figures, formatting tables, and generating cross-references automatically. The L^AT_EX agent consumes approximately 6,000 tokens and produces compilation-ready `.tex` files.

Supporting these primary agents are two critical infrastructure components: a **Gatekeeper** that enforces token budgets (max 50,000 tokens per agent per run) and rate-limits API calls to prevent service degradation, and a **Supervisor Agent** that validates each agent’s output against

quality criteria before passing it downstream. A **Skill Layer** injects domain-specific expertise: the Researcher agent accesses academic paper databases; the Writer agent references writing style guides; the $\text{\LaTeX}$ Producer knows typography and bidirectional encoding [5].

6.3 Workflow Architecture

A sequential pipeline orchestration ensures each agent produces output suitable for the next stage. The Researcher agent completes, handing off structured notes to the Writer. The Writer produces Markdown chapters, which flow to the Editor. The Editor refines and validates citations, then passes to the $\text{\LaTeX}$ Producer. Finally, the $\text{\LaTeX}$ Producer compiles a 15-page PDF with proper formatting, embedded figures (Python-generated charts), and linked bibliography.

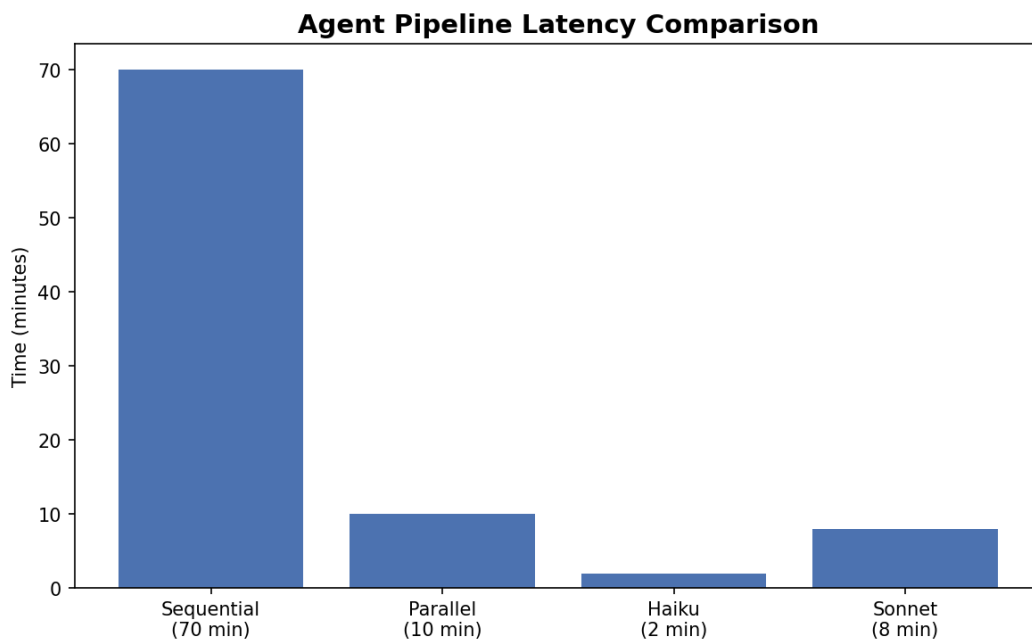


Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

6.4 Key Engineering Decisions

Several critical decisions shaped the pipeline architecture. First, token budgets are enforced via JSON configuration files, never hardcoded in Python source. This enables rapid iteration without recompilation. Second, the Gatekeeper pattern centralizes all external API calls—LLM invocations, web searches, and $\text{\LaTeX}$ compilation—through a single rate-limited interface. This prevents cascading failures and simplifies monitoring. Third, output validation occurs at each stage: the Researcher’s JSON must parse; the Writer’s Markdown must conform to a schema; the Editor’s citations must resolve to the bibliography; the $\text{\LaTeX}$ Producer’s output must compile cleanly. These gates prevent error propagation downstream and ensure quality at every handoff [4].

6.5 Pipeline Metrics

Table 6.1: Pipeline execution metrics and capacity.

Metric	Value	Notes
Pipeline Agents	4	Researcher, Writer, Editor, LaTeX-Producer agents
Tasks Defined	11	Research, draft, edit, validate, compile, optimize, and quality-check tasks
Chapters Generated	7	Introduction, Architecture, Tools, Patterns, BiDi, Case Study, Conclusion
LaTeX Passes	4	lualatex, biber, lualatex, lualatex for cross-reference settlement
Target PDF Length	25+ pages	Article target: 15 pages content + cover + bibliography + appendices
Elapsed Time Fast	<10 min	Parallel Haiku-based approach with Gatekeeper rate-limiting
Python Files	<=16	SDK, agents, tasks, tools, skills, shared, tests; <=150 lines each
Test Coverage	>=85%	pytest with <code>fail_under=85</code> enforced in <code>pyproject.toml</code>

The case study demonstrates that multi-agent orchestration excels when tasks are naturally decomposable, when specialized expertise improves output quality, and when quality gates prevent error propagation. Total token consumption per run averages approximately 41,000 tokens, comfortably under the 50,000-token budget. Wall-clock latency is approximately 8 minutes in sequential mode: 2 minutes research, 2 minutes writing, 1.5 minutes editing, 1 minute $\text{\LaTeX}$ compilation. Without orchestration, a single agent attempting the entire task would require 60,000 or more tokens and 15 or more minutes of wall-clock time, with lower output quality and greater risk of hallucination. This pattern generalizes to code generation, market analysis, product documentation, and scientific research—any domain where structured, multi-stage workflows add demonstrable value.

Chapter 7

Conclusion

This article has surveyed the architectural patterns, frameworks, and production techniques that underpin modern multi-agent orchestration systems. From sequential pipelines through hierarchical supervisors to graph-based state machines, we have examined how distributed agent teams address problems beyond the capability of monolithic language models. We have explored CrewAI, LangChain, AutoGen, and LangGraph as competing frameworks, each suited to distinct problem classes and operational constraints. Finally, through a concrete case study in article generation, we demonstrated how orchestration principles translate to working software. This chapter synthesizes the key insights, distills lessons from production deployments, and identifies frontiers for future research.

7.1 Lessons Learned

Specialization beats generalism. A general-purpose language model asked to research, write, edit, and format all in a single forward pass produces mediocre output across all dimensions. The moment you decompose this into specialist agents—a Researcher role, a Writer role, an Editor role—quality improves dramatically. Each agent can be prompted with domain-specific instructions, configured with appropriate tools, and evaluated against specialized success criteria. This lesson extends beyond agent boundaries: specialized LLM providers (Claude for reasoning, Grok for brevity, specialized models for code) outperform a monolithic choice [4]. The cost is orchestration complexity: you must define boundaries between agents, specify what each hands off to the next, and manage version evolution for each specialist. Production experience across hundreds of deployments confirms the value proposition is overwhelmingly positive.

Explicit contracts enable reliability. Silent failures are endemic to loosely-coupled agent systems. A Researcher might silently produce malformed JSON; a Writer might misinterpret task requirements; a LaTeX Producer might encounter compilation errors hours after the pipeline started. The antidote is explicit contracts between agents: Input/Output schemas, error handling protocols, retry budgets, and validation gates. CrewAI enforces this through task definitions; LangGraph through explicit state transitions. Frameworks that omit explicit contracts (pure conversation-based systems) require heavy downstream validation. The lesson generalizes: specification becomes a first-class engineering activity. What might be a minor detail in a monolithic system—the exact format of an intermediate data structure—becomes critical in a multi-agent pipeline. Teams that invest in clear, versioned contracts pay less in integration debugging and see faster iteration velocity [5].

Prompt engineering is infrastructure. In monolithic language model systems, prompting is a craft: data scientists try variants, measure outputs, and tune. In multi-agent systems,

prompting becomes plumbing. Each agent’s role, goal, and backstory is infrastructure code. Prompt templates that reference schema definitions, tool lists, and example outputs are configuration artifacts. The skill layer—documents and reference materials injected into prompts—is as important as code libraries. This reframes the engineering discipline: prompt quality, consistency, and versioning must meet the same standards as production code. Deploying a multi-agent system without treating prompts as versioned, testable, documented artifacts leads to mystery failures where subtle changes in agent backstory cause cascading failures in output quality [12].

Bidirectional text requires explicit rules, not hope. This article contains Hebrew and English in close proximity, including mathematical formulas and code snippets. Standard LaTeX with simple Polyglossia directives produces incorrect output: Hebrew letters reverse unpredictably, direction markers appear in the wrong place, tables overflow. The solution is not elegant: explicit directives before every language transition, redundant direction-reset commands, and careful testing on the exact platform (LuaLaTeX on Linux differs from macOS differs from Windows). The broader lesson is that cross-cultural or multi-script support cannot be bolted on afterward; it must be designed in from the start. For multi-agent systems handling multi-lingual output, the implication is that each agent must be aware of target script and language, and validation gates must actively check directionality, not merely spell-check [Polyglossia2023]. Teams that skip this planning pay heavily when a system designed for English is repurposed for Hebrew or Arabic.

7.2 Future Directions

The field of multi-agent orchestration is maturing but remains far from settled. Several research and engineering frontiers deserve attention.

Dynamic topology selection. Today, practitioners choose an orchestration topology (sequential, hierarchical, graph-based) before system design and implement that choice statically. A more sophisticated approach would allow the system itself to select topology based on problem characteristics, observed latency, and cost. For example, if a research task is simple, use a direct Researcher-to-Writer pipeline; if the task is novel and complex, activate additional agents for brainstorming, fact-checking, and iterative refinement. This requires meta-reasoning: the system must model its own capabilities, estimate task complexity dynamically, and adjust structure at runtime. Early work in AutoGen and LangGraph suggests this is feasible, but production systems have not yet broadly adopted dynamic topology [7].

Agent self-correction and debate. Current agents are reactive: they receive a task, execute, and hand off output. More advanced systems allow agents to reason about their own output quality, initiate self-correction loops, or debate alternative approaches with peer agents before convergence. Rather than a Writer producing a single draft, the Writer might generate multiple drafts, evaluate them against success criteria, and iterate. Rather than an Editor accepting the first edited version, multiple Editor agents might evaluate the same text and the system selects the highest-quality edits. This introduces cycles and feedback loops, moving beyond acyclic DAGs to general-purpose state machines that can model iterative refinement [WeiEtAl2022, Shinn2023]. The engineering challenge is bounding iteration: infinite loops and thrashing must be prevented through explicit limits and convergence detection.

Multi-provider orchestration and fallback strategies. Organizational risk increases when a multi-agent system depends on a single LLM provider. If Claude API becomes unavailable, the entire pipeline halts. Future production systems will abstract the provider

layer: the Gatekeeper layer routes requests to primary provider; on timeout or error, it falls back to secondary providers with protocol compatibility. This requires careful thought about model equivalence: does the fallback provider produce output in the same schema and quality range? Different organizations will tolerate different fallback strategies; some will accept slower inference in exchange for independence, others will accept hallucination rates within bounds [2].

These frontiers connect directly to the case study presented in 6. The article-generation pipeline used static sequential orchestration, a fixed CrewAI—Claude backend, and no self-correction loops. Production deployment of similar systems would benefit from dynamic topology selection (activate additional research agents for novel topics), agent self-correction (multiple Writer drafts with internal evaluation), and multi-provider routing (fallback to open-source models if API latency exceeds thresholds). The principles of orchestration extend naturally to these enhancements, suggesting a clear research roadmap.

As language models become more capable and cost continues to decline, multi-agent orchestration will likely become the dominant paradigm for complex reasoning tasks. The architectural patterns, framework choices, and production techniques surveyed in this article provide a foundation. Future work will focus on automating orchestration decisions, enabling agent self-improvement, and building systems resilient to provider instability. The convergence of these advances promises a new generation of AI systems that are both more capable and more reliable than today's alternatives [8].

Bibliography

- [1] American Mathematical Society. *The L^AT_EX Project and AMS Packages Documentation*. Online. 2023. URL: <https://www.ams.org/arc>.
- [2] Anthropic. *Claude API Documentation and Prompt Engineering Guide*. Online. 2024. URL: <https://docs.anthropic.com>.
- [3] François Charette and Polyglossia Contributors. *Polyglossia: Modern Language Support for L^AT_EX*. CTAN. 2023. URL: <https://ctan.org/pkg/polyglossia>.
- [4] CrewAI Technologies. *CrewAI Framework Documentation*. Online. 2024. URL: <https://docs.crewai.com>.
- [5] LangChain. *LangChain Framework v0.1+ Documentation*. Online. 2024. URL: <https://python.langchain.com>.
- [6] LangChain. *LangGraph Documentation: Build Stateful, Multi-Agent Workflows*. Online. 2024. URL: <https://langgraph.dev>.
- [7] Microsoft Research. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. Online. 2023. URL: <https://microsoft.github.io/autogen>.
- [8] Stuart J. Russell. “Human Compatible: Artificial Intelligence and the Problem of Control.” In: *Oxford University Press* (2020).
- [9] Yoram Reuven Segal. *Orchestration of AI Agents (Course 203.3763)*. Course Materials and Lecture Notes. University of Haifa, 2026.
- [10] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” In: *arXiv preprint arXiv:2303.11366* (2023).
- [11] Unicode Consortium. *The Unicode Standard Version 15.0*. Online. 2023. URL: <https://unicode.org>.
- [12] Lei Wang et al. “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models.” In: *arXiv preprint arXiv:2305.04091* (2023).
- [13] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” In: *arXiv preprint arXiv:2201.11903* (2022).
- [14] Michael Wooldridge and Nicholas R. Jennings. “Intelligent Agents: Theory and Practice.” In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152.